

Toxic Twitter Comment Classification System — Implementation Plan

Based on the SRS (Anirudh Thiagarajan Trichy, Antony Thaikadavil, KV Akash — NITK IT).

1. Architecture & Stack (assumed defaults — adjust if your course mandates something else)

The SRS itself defines four logical components — that maps cleanly onto a 3-person team plus a shared data layer:

Component	Owns
Backend Service (Person A)	Auth, 2FA, sessions, activity logging, admin user management, DB schema
Feed & Classification Service (Person B)	Twitter/X API integration, ML toxicity classifier, confidence scoring
Web Client (Person C)	All 7 UI views incl. Admin Dashboard, consumes both backend APIs

Suggested stack:

Frontend: React + Vite, Tailwind, Axios

Backend API: Node.js + Express (or FastAPI if the team is more Python-comfortable) + JWT auth + PostgreSQL

Classification service: Python + FastAPI, scikit-learn or a fine-tuned DistilBERT on the Jigsaw dataset, exposed as an internal REST endpoint

DB: PostgreSQL (Supabase or Neon free tier gives you a hosted instance immediately — good for 3 people working in parallel without a shared local DB)

2FA: TOTP (e.g. `speakeasy`/`pyotp` + an authenticator app) is simpler to demo than SMS/email OTP — no external SMS provider needed

Deployment: Frontend → Vercel/Netlify; Backend API + Classification service → Render/Railway (Docker); DB → Supabase/Neon

Critical first step: before anyone writes feature code, agree on the **API contract** (below) so all three people can build against a stub without blocking each other.

2. Shared API Contract (lock this in Phase 0)

```
POST /api/auth/register      {email, password}
POST /api/auth/login        {email, password} -> {tempToken}
POST /api/auth/2fa/verify    {tempToken, code} -> {jwt}
POST /api/admin/login        {email, password} -> {tempToken}
GET  /api/logs?user=&date=&action= [admin only] -> [logEntry]
GET  /api/admin/users        [admin only] -> [user]
PATCH /api/admin/users/:id    [admin only] {status} -> user
```

```
POST /api/feed/fetch        [auth required] -> {jobId} or [comment]
GET  /api/feed/history      [auth required] -> [pastFetch]
```

Internal, called by backend, not exposed to frontend directly:

```
POST /classify (classification service) {text} -> {label, subcategories[], confidence}
```

Every one of these calls must write an entry to the activity log (F.10/NF.4) — build the logging as **middleware**, not something each route remembers to call.

3. Phase-by-Phase Plan

Each phase below has a ready-to-paste prompt for your AI coding agent (Claude Code, Cursor, etc.). Run phases 1–3 in parallel across the three of you once Phase 0 is done; phases 4+ need integration.

Phase 0 — Project Setup & Contracts (all 3, together, ~1 session)

Goal: repo, environments, DB schema skeleton, API contract file, task board.

AI agent prompt:

Set up a monorepo for a "Toxic Twitter Comment Classification System" with three packages: /web-client (React + Vite + Tailwind), /backend (Node.js + Express + PostgreSQL via Prisma or Sequelize), /classifier-service (Python + FastAPI). Add a root docker-compose.yml that runs all three plus a Postgres container. Create .env.example files for each package listing required vars (DB_URL, JWT_SECRET, TWITTER_API keys, CLASSIFIER_SERVICE_URL). Add a README with setup instructions. Create the initial Prisma/Sequelize schema for: users(id, email, password_hash, role[user|admin], status, totp_secret, created_at), activity_logs(id, actor_id, actor_type, action_type, details_json, timestamp), feed_fetches(id, user_id, fetched_at), classifications(id, fetch_id, comment_text, label, subcategories_json, confidence, timestamp). Do not implement business logic yet — just scaffolding, schema, and health-check endpoints for each service.

Deliverable: repo pushed, everyone can `docker-compose up` and hit a `/health` endpoint on all three services.

Phase 1 — Backend: Auth & 2FA (Person A) — F.1-F.5, NF.2, C.3

AI agent prompt:

☐

In /backend, implement:

1. POST /api/auth/register — validate email format, password strength, hash password with bcrypt, create user with role=user, status=active.
2. POST /api/auth/login and POST /api/admin/login — verify credentials, issue a short-lived tempToken (not a full session) pending 2FA.
3. TOTP-based 2FA: on registration generate a TOTP secret (use `speakeasy` or `otplib`), return a QR-code provisioning URI for the user to scan into an authenticator app. POST /api/auth/2fa/verify — validate the 6-digit code against the stored secret, issue a JWT (24h expiry) on success.
4. Auth middleware that verifies JWT and attaches req.user, plus a separate requireAdmin middleware.
5. Admin accounts must NOT be creatable via the public register endpoint — add a seed script (npm run seed:admin) that creates one from env vars.
6. Never log or return plaintext passwords in any response or log entry.

Write integration tests for each endpoint (happy path + invalid credentials + invalid/expired 2FA code).

Phase 2 — Backend: Activity Logging Middleware (Person A) — F.10, NF.4, C.4

AI agent prompt:

☐

In /backend, implement Express middleware that automatically writes an activity_logs row for every request to a protected route: capture actor id, actor type (user/admin), action type (derived from route+method), a details JSON blob (route params, outcome), and timestamp. Wire this middleware globally so no route can be added later without being logged — fail closed: if the log write fails, the request should fail too, since "no significant transaction may occur without an audit entry" (SRS C.4). Add GET /api/logs (admin-only) with optional filters ?user=&date=&action= and pagination. Add tests confirming a failed login attempt, a successful login, a feed fetch, and an admin action all produce a log entry.

Phase 3 — Classification Service (Person B) — F.7, F.8, F.9

AI agent prompt:



In /classifier-service (FastAPI):

1. Build a text preprocessing pipeline (lowercase, strip URLs/mentions, tokenize).
 2. Train or fine-tune a toxicity classifier on the Jigsaw Toxic Comment Classification dataset (multi-label: toxic, severe_toxic, obscene, threat, insult, identity_hate). Start with a TF-IDF + Logistic Regression baseline per-label (fast to train, good enough for a course demo) before attempting a transformer fine-tune if time allows.
 3. Expose POST /classify {text: string} -> {label: "toxic"|"non-toxic", subcategories: string[], confidence: float}. Confidence = max predicted probability across labels.
 4. Response time must be well under the 3-second budget in NF1 — benchmark and cache the loaded model in memory at startup, don't reload per request.
 5. Add a /health endpoint and a small test script that runs 10 known toxic and 10 known clean sample comments through /classify and prints results.
- Save the trained model artifact to /classifier-service/model/ and document how to retrain it in the README.

Phase 4 — Backend: Feed Retrieval + Integration Glue (Person B) — F.7

AI agent prompt:



In /backend, implement POST /api/feed/fetch (auth required):

1. Call the Twitter/X API to retrieve recent comments/replies from the single designated dummy account (use env vars for API keys/account id — never hardcode). Cache/rate-limit calls to avoid hitting API limits during demos.
 2. For each retrieved comment, call the classifier-service's POST /classify, and persist the comment text + classification result + confidence to the classifications table, linked to a feed_fetches row for this user+timestamp.
 3. Return the full set of classified comments to the caller.
 4. Add GET /api/feed/history (auth required) returning the calling user's own past fetches with their classification results (User Activity History view, SRS 3.2.1).
 5. Handle Twitter/X API failure gracefully (timeout, rate limit) with a clear error the frontend can display — do not let it hang past a few seconds.
- Add a mock mode (env flag) that returns canned sample comments instead of calling the real Twitter/X API, so Person C can build the frontend without depending on live API keys.

Phase 5 — Frontend: Public + Auth Views (Person C) — F.6, 3.2.1

AI agent prompt:

 In /web-client, build against the API contract in API_CONTRACT.md (use a mock server or MSW if backend isn't ready yet):

1. Landing Page — public, describes the system, links to Register/Login.
2. Registration Page — email/password/confirm, inline validation, shows the TOTP QR code returned by the backend after successful registration.
3. Login Page (separate flows for user vs admin, same visual pattern) -> Two-Factor Verification Page (6-digit code entry) -> on success, store the JWT and redirect to the Homepage.
4. Global error display pattern for failed login / expired 2FA / validation errors — plain, non-technical language per NF.5.

Use React Router for navigation and a shared Axios instance that attaches the JWT and handles 401s by redirecting to login.

Phase 6 — Frontend: Classification & History Views (Person C) — F.7-F.9, 3.2.1

AI agent prompt:

 In /web-client, build:

1. Homepage / Classification Page — a "Fetch Feed" button that calls POST /api/feed/fetch, shows a loading state, then renders each returned comment with its toxic/non-toxic label, sub-category tags if toxic, and confidence score (e.g. as a percentage badge). Clear visual distinction between toxic and non-toxic results.
2. User Activity History View — calls GET /api/feed/history, lists past fetches chronologically, each expandable to show that fetch's results.

Handle empty states (no fetches yet) and error states (feed fetch failed) per NF.5 usability requirements.

Phase 7 — Frontend + Backend: Admin Dashboard (Person C + Person A) — F.11, F.12

AI agent prompt:



Backend (Person A) already exposes GET /api/logs and GET/PATCH /api/admin/users. In /web-client, build the Admin Dashboard:

1. System-wide activity log view with filter controls (user, date range, action type) and pagination, calling GET /api/logs.
2. User management table — list all users with status, and an activate/deactivate action calling PATCH /api/admin/users/:id.

Gate this whole route behind a check that the logged-in JWT has role=admin; redirect non-admins away.

Phase 8 — Integration & System Testing (all 3, together)

AI agent prompt:

 Run the full docker-compose stack. Write an end-to-end test script (Playwright or Cypress) that: registers a user, completes 2FA, fetches the feed, verifies classification results render, checks the activity log recorded every step (as an admin), and verifies a non-admin is blocked from the admin dashboard. Also verify: no plaintext password ever appears in logs or API responses; a comment submitted for classification returns a result within 3 seconds (NF.1); the Twitter/X mock mode works without live API keys for demo safety. Fix any integration bugs surfaced (mismatched field names, CORS, auth header issues) — these are the most common failure points when three people build against a shared contract independently.

Phase 9 — Deployment (all 3, together)

AI agent prompt:



Prepare this project for deployment:

1. Add production Dockerfiles for /backend and /classifier-service (multi-stage builds, non-root user, no dev dependencies in the final image).
 2. Add a vercel.json or netlify.toml for /web-client with the production API base URL as an env var, not hardcoded.
 3. Write deployment steps for: DB (Supabase/Neon — run migrations), classifier-service and backend (Render/Railway — set env vars, health checks, restart policy), frontend (Vercel/Netlify — connect repo, set env vars).
 4. Ensure CORS on the backend only allows the deployed frontend origin.
 5. Ensure all secrets (JWT_SECRET, Twitter API keys, DB URL) are set via the platform's env var UI, never committed to the repo — add a check that .env files are gitignored.
 6. Add a smoke-test checklist to run after deploy: register, login+2FA, fetch feed, view history, admin login, view logs — confirm each against the live URLs.
- Output the final set of live URLs and a one-page runbook for restarting/redeploying each service if something breaks during evaluation/demo.

4. Suggested Timeline (adjust to your course deadline)

Week	Focus
1	Phase 0 (setup) + Phases 1–3 start in parallel
2	Finish Phases 1–4, start Phase 5
3	Phases 5–7
4	Phase 8 (integration testing) + bug fixing
5	Phase 9 (deployment) + demo prep, buffer for issues

5. Coordination Tips for a 3-Person Split

Daily 10-min sync — since Person C's frontend depends on both backend APIs, mismatches (field name typos, unexpected nulls) are the #1 time-sink. Catch them early.

Mock mode is not optional — Person C should never be blocked waiting for Person B's live Twitter/X integration; the mock-mode flag in Phase 4 exists specifically so all three can work in parallel.

One person (suggest Person A, since they own the DB schema) owns migrations — everyone else proposes schema changes as PRs rather than editing the DB directly, to avoid conflicting migrations.

Feature branches + PR review across the three of you, even on a small team — it's the cheapest way to catch integration issues before Phase 8.